NATIONAL AI HACKATHON 2026 · SCHUBKI · **MAKING SENSE OF LOGS**

One corpus.
Three roles.
Every answer has a receipt.

Just under 5 million real log lines from a supercomputer and a cloud platform. An engineer, a team lead, and a manager each need a different answer from the same logs — and most AI tools over logs either dump raw text back at you or confidently invent the parts they don't know.

◆ Engineer

What failed, in which component, and what happened just before and after?

◆ Team Lead

Which problems recur, how often, and which team owns them?

◆ Manager

Which observed problems deserve attention first — with no invented impact?

THE IDEA

The database searches and calculates. Claude explains. Every number traces to a query that actually ran.

Incident Lens is an MCP server — 13 parameterized tools over BGL and OpenStack logs in SereneDB. Claude never writes SQL; the server does all searching and counting, and writes every call to a hash-chained evidence ledger before returning. Any Claude — yours, a judge's, anyone's — can connect and use it with their own account, from a terminal, Claude Desktop, or a web console built for this.

```
◆ E-0097  tool: search_logs

scope: bgl · 2005-06-01 → 2006-02-01
filters: query="invalid or missing
  program image", mode=phrase
total_matches: 112584  truncated: true
ledger_hash: 97633d5663bf9ef0…
note: "quote frequency only from
  total_matches, never len(rows)"
```

ARCHITECTURE

Claude sees only what the server returns



The read-only role `il_reader` can never write; every tool call is logged before it is returned, so any number a judge sees can be replayed with `get_evidence` and independently re-verified with `verify_ledger_integrity`.

ONE REAL ISSUE · THREE LENSES · BGL

ciod job-launch failures — not a textbook error, a real discovery

A parallel job's executable path is missing or unreadable — found, traced, and quantified by the same three tool paths each role would actually use.

◆ Engineer

First hit: job 65641 ("raptor") fails on **1,024 distinct nodes** within milliseconds of each other — not one bad node.

`search_logs → nearby_events → count_by`

◆ Team Lead

112,584 entries, 52,763 episodes. One week (2005-08-08) alone jumps **+9,300%**. Owner: Application Support.

`trend → episodes → count_by(team)`

◆ Manager

Still **accelerating** (+224% in the last 4 weeks) while two other top-ranked problems dropped to zero. Priority score: **19,502** vs 0 vs 0.

`corpus_overview → recurring_patterns`

◆ Blind-LLM check

Asked the identical question, a fresh Claude with **no tools and no data access** confidently named **memory/ECC errors on KERNEL** as the top cause, with invented frequency numbers — then admitted, unprompted, that every specific claim was "a pure guess." Wrong component, wrong category, wrong everything except the fluency.

A SECOND, DELIBERATELY DIFFERENT SHAPE · OPENSTACK

The methodology isn't a one-dataset trick

A nova-compute periodic task (image-cache verification) throws and aborts **entirely** whenever it hits one already-deleted instance's disk — a real code-level gap, not an environmental blip.

8

occurrences, over ~26 hours

8

separate episodes — entries = episodes, the opposite shape from the BGL burst above

8

different VM instances — never the same one twice

#1

most frequent ERROR-level template in the whole OpenStack corpus

Same tool-calling loop, same four-block answer contract, same evidence ledger — run against a completely different system with a completely different failure shape. `investigation/openstack_investigation.md`, checked clean against the citation checker.

NOVELTY · A TOOL THE HACKATHON DIDN'T ASK FOR

What a frequency ranking hides, on purpose

`concentration_anomalies` ranks failures by how **confined** they are to one node or service — not how often they fire. The two rankings disagree, and the disagreement is the finding.

◆ Buried by frequency

`MACHINE CHECK DCR read timeout` — **14,481 entries, confined to exactly 1 node** out of the whole system. Ranked **#33** by raw count: invisible past the default view.

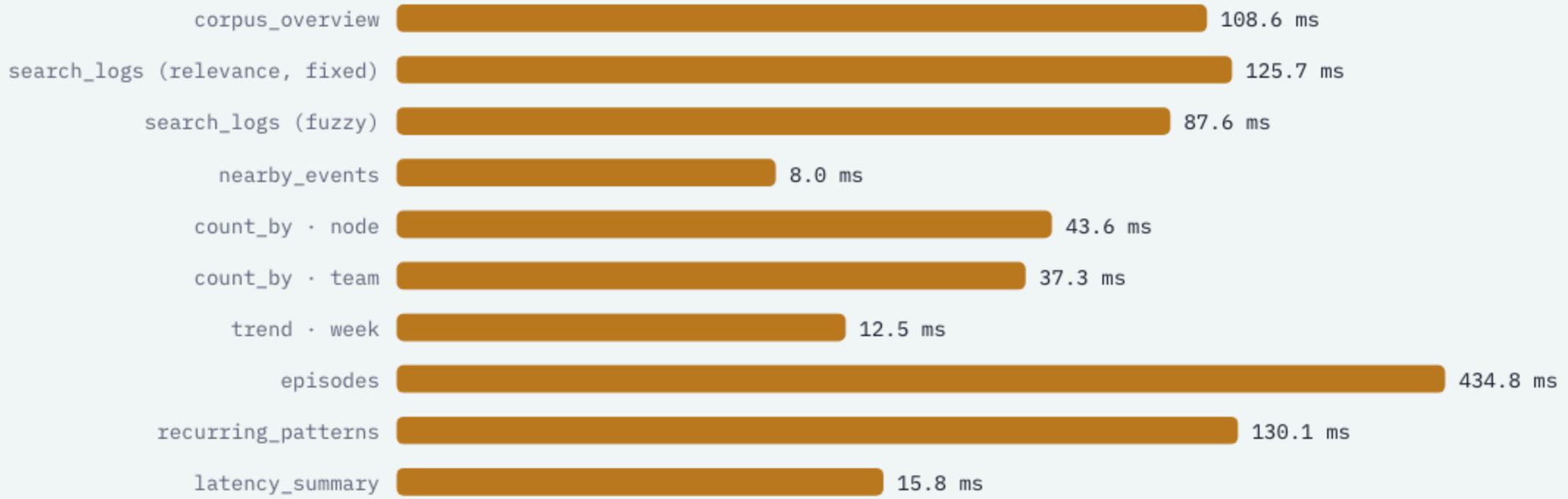
◆ Hidden in plain sight

`idoproxydb hit ASSERT condition` — **84,168 entries, 1 node**. The single most concentrated failure in the whole corpus; a human scrolling a frequency list would never isolate it this cleanly.

A high ratio says "go check that one node's hardware first." A low ratio says "this is probably systemic." Both are exact counts — no ML, no guessing.

SERENEDB AT SCALE

4,955,757 rows. Every number below actually scanned them.



Load + index the full corpus: **3.4–9.2s** tables, **5.9–9.6s** inverted indexes, across repeated runs on a laptop — no pre-aggregation, no caching tricks.

The one outlier we found (`search_logs`, relevance mode) was **root-caused and fixed**, not hidden: a redundant time filter was silently disabling the index's fast top-k scoring path. Dropping the provably-redundant filter turned a **~1.5s** query into **~150–215ms** — same result, 9x faster.

THE EVIDENCE LEDGER & ANSWER CONTRACT

Four blocks. Every number carries a receipt — and the receipt book is tamper-evident.

◆ 1 · Observed

Measured facts only — scope, period, filters, `[E-id]`. Never `len(rows)` as a frequency.

◆ 2 · Hypotheses

Possible explanations, explicitly labelled. Never stated as fact.

◆ 3 · Recommended checks

Specific, ordered, each tied to an evidence id.

◆ 4 · Unknown

What the logs don't establish — cause, downtime, cost, customer impact. Never invented.

◆ Tamper-evident, not just append-only

Every ledger entry's hash is `sha256(previous entry's hash + this entry's own content)`. Hand-edit one stored number with a raw SQL UPDATE — bypassing every tool — and `verify_ledger_integrity` catches it and names the exact entry. Proven live in the web console's "Simulate Tamper" button, against a scratch copy that never touches the real ledger.

A BROWSER YOU CAN WATCH IT THINK IN

The same 13 tools, with the evidence ledger made visible

Chat with Claude through the browser — same role prompts, same tools, called through the real Anthropic API. Every tool call lands on the right as it happens: the exact SQL, the exact count, the ledger hash, chained visibly to the one before it. Click any evidence id to replay it.

Not a second implementation — it's a thin UI over the exact same server. Tested over the real HTTP surface (overview, ledger status, tamper demo, evidence replay, path-traversal blocked), and the chat loop itself is driven end-to-end by a fake-client test harness, so the only genuinely untested piece is the literal network call to Anthropic — which needs a real key to exercise once, same as any MCP client.

BONUS · DATABASE ROW-LEVEL SECURITY

Per-team access, enforced by the database, not application code

SereneDB has no native `ROW LEVEL SECURITY` / `CREATE POLICY` — checked directly, not assumed. Built the same real guarantee from primitives it does support: one filtered view per team, straight from the ownership table, with that team's base-table grants revoked.

9

real teams provisioned, straight from the ownership table

0

rows any team's role can see outside its own component/service

✓

verified by connecting as each restricted role — never as admin

SCOREBOARD

What's actually been run, not just written

13
MCP tools, all wired and tested

39
pytest tests against plain-SQL ground truth

4.9M
real log rows loaded, indexed, and queried

15+
real bugs found and fixed by testing at full scale, not assumed away

2
full worked investigations, both datasets, both clean against the citation checker

9
per-team database roles, each verified in isolation

1
clean-checkout run, fresh venv to passing test suite, zero manual fixes

0
invented financial, downtime, or customer-impact claims

LIMITS & WHAT'S NEXT

Honest about what the logs can't tell you

◆ Does not establish

Root cause of any single job's bad path · who submitted the spike · downtime caused · financial or customer impact.

◆ Next up

A larger Loghub dataset for the scale award · live golden-set run against the real Claude API · demo recording.

---

github.com/ApparentlyTejas/incident-lens

One setup command. Clean checkout to first answered question in minutes — verified, not assumed.